

MODELLI E ARCHITETTURE AVANZATI DI BASI DI DATI

Alessandro Zappatore

Università degli studi di Torino

Contents

1	Data intensive	4
1.1	OLTP (Online Transaction Processing)	4
1.2	OLAP (Online Analytic Processing)	4
1.3	Data Warehouse	4
1.3.1	ETL	4
1.4	Data Lake	4
1.4.1	Sushi Principle	4
1.5	Servizi cloud	4
1.6	Motivazioni sistemi distribuiti	5
1.7	Reliability	5
1.7.1	Fault	5
1.7.2	Failure	5
1.8	Tempi	5
1.8.1	Response time	5
1.8.2	Service time	5
1.8.3	Queueing delays	5
1.8.4	Latency	5
1.8.5	Throughput	5
1.8.6	X-esimo percentile = Y ms	5
2	Data Models	6
2.1	Modello Relazionale	6
2.2	Modello a Documenti	6
2.3	Modello a Grafo	6
2.4	Schemi Analitici (Data Warehouse)	6
2.5	Altri Modelli (NoSQL)	7
3	Replication	8
3.1	Single-Leader Replication (Active/Passive)	8
3.2	Handling Node Outages	8
3.3	Replication Lag e Consistenza	8
3.4	Multi-Leader Replication	9
3.5	Leaderless Replication	9
3.6	Detecting Concurrent Writes	9

4	Sharding	11
4.1	Strategie di Partizionamento	11
4.2	Workload Sbilanciati e Hot Spot	11
4.3	Rebalancing (Ribilanciamento)	11
4.4	Request Routing (Service Discovery)	11
4.5	Secondary Indexes nello Sharding	12
5	Consistency (Consistenza e Consenso)	13
5.1	Eventual Consistency vs Strong Consistency	13
5.2	Linearizability (Consistenza Forte)	13
5.3	Il Teorema CAP	13
5.4	Algoritmi di Consenso	13
5.5	Serializzabilità vs Linearizzabilità	14
5.6	Distributed Transactions e Two-Phase Commit (2PC)	14
6	Batch Processing	15
6.1	Caratteristiche Principali	15
6.2	L'Analogia con Unix e i Sistemi Distribuiti	15
6.3	Core Orchestrator Components	15
6.4	Modello MapReduce	15
6.5	Modello Dataflow (Apache Spark, Flink)	16
7	Stream Processing	17
7.1	Eventi e Message Brokers	17
7.2	Change Data Capture (CDC) e Log Compaction	17
7.3	Command Query Responsibility Segregation (CQRS)	17
7.4	Casi d'Uso e Join negli Stream	18
7.5	Tolleranza ai Guasti (Fault Tolerance)	18
8	MongoDB: Architettura e Query	19
8.1	Replication (Replica Sets)	19
8.2	Sharding (Scalabilità Orizzontale)	19
8.3	Tipi di Query in MongoDB	19
8.3.1	Query di Lettura (find)	20
8.3.2	Pipeline di Aggregazione (aggregate)	20
8.3.3	Query di Aggiornamento e Cancellazione	20
8.4	Funzionalità Avanzate	21
9	Neo4j e Database a Grafo	22
9.1	Property Graph Model (Modello a Grafo di Proprietà)	22
9.2	Cypher Query Language	22
9.3	Architettura e Transazioni	22
9.4	Modellazione e Schema	22
9.5	Il Pattern "Thin Node" (Polyglot Persistence)	23
10	Amazon DynamoDB	24
10.1	Modello dei Dati e Chiavi	24
10.2	Partizionamento e Problema degli Hot Spot	24
10.3	Lectture e Consistenza	24
10.4	Indici Secondari	24
10.5	Query, Scan e Limiti Architeturali	25
10.6	Global Tables e Gestione della Capacità	25

1 Data intensive

Salvare e processare grandi volumi di dati, gestendo cambiamenti, garantendo consistenza e facendo sì che i servizi siano altamente disponibili.

1.1 OLTP (Online Transaction Processing)

Backend service e infrastruttura dove i dati sono creati.

Tipicamente guarda una piccola quantità di record.

- Insert
- Update
- Delete

1.2 OLAP (Online Analytic Processing)

Usato da business analyst e data scientist.

Copia dei dati **read-only**.

Ottimizzato per operazioni di analisi.

1.3 Data Warehouse

Database separato per query analitiche

Copia read-only dei dati nei vari sistemi OLTP.

1.3.1 ETL

- Extract
- Transform
- Load

1.4 Data Lake

Repository centralizzata che contiene tutti i dati che potrebbero essere utili per l'analisi.

Contiene **raw-data**.

1.4.1 Sushi Principle

Raw data is better

Possono essere trasformati all'evenienza.

1.5 Servizi cloud

- **IaaS** (Infrastructure): Macchina virtuale
- **PaaS** (Platform): Macchina virtuale e software
- **SaaS** (Software): Applicazione ready to use
- **FaaS** (Function): Funzione specifica

1.6 Motivazioni sistemi distribuiti

- Tolleranza ai guasti e Disponibilità
- Scalabilità
- Latenza e Geografia
- Elasticità
- Hardware specializzato
- Conformità legale
- Sostenibilità

1.7 Reliability

1.7.1 Fault

Guasto: Quando un singolo componente del sistema devia dalle sue specifiche.

1.7.2 Failure

Fallimento: Quando il sistema nel suo complesso smette di funzionare:

- Hardware fault
- Software fault
- Errore umano

1.8 Tempi

1.8.1 Response time

Tempo totale percepito dal client dal momento della richiesta.

1.8.2 Service time

Tempo in cui elabora la richiesta.

1.8.3 Queueing delays

Tempo di attesa in coda.

1.8.4 Latency

Tempo attraversamento rete

1.8.5 Throughput

Operazioni al secondo.

1.8.6 X-esimo percentile = Y ms

$X\%$ delle richieste viene elaborato in meno di Y millisecondi.

2 Data Models

Un data model definisce come le informazioni sono strutturate, archiviate e recuperate. È la parte più importante del database perché determina come gli sviluppatori pensano ai dati e influenza direttamente le performance e la flessibilità del sistema.

2.1 Modello Relazionale

Organizza i dati in relazioni (tabelle) contenenti tuple (righe)

- Dati **normalizzati**: si evitano le duplicazioni usando riferimenti tramite chiavi esterne (foreign key)
- Ottimale per gestire relazioni **molti-a-uno** e **molti-a-molti**
- **Impedance mismatch**: problema causato dalla differenza tra i linguaggi di programmazione orientati agli oggetti e le tabelle relazionali, che richiede un livello di traduzione (es framework ORM)

2.2 Modello a Documenti

I dati sono memorizzati in documenti auto-contenuti, spesso in formato JSON

- **Schema flessibile** (schemaless): documenti nella stessa collezione possono avere campi diversi, agevolando l'evoluzione dell'applicazione
- Ideale per relazioni **uno-a-molti** e dati gerarchici, garantendo una migliore "località" del dato
- Incoraggia la **denormalizzazione** (duplicazione dei dati) poiché i JOIN nativi sono deboli o assenti, costringendo l'applicazione a risolverli manualmente idratando gli ID
- La denormalizzazione velocizza le letture ma rende le scritture più complesse e costose

2.3 Modello a Grafo

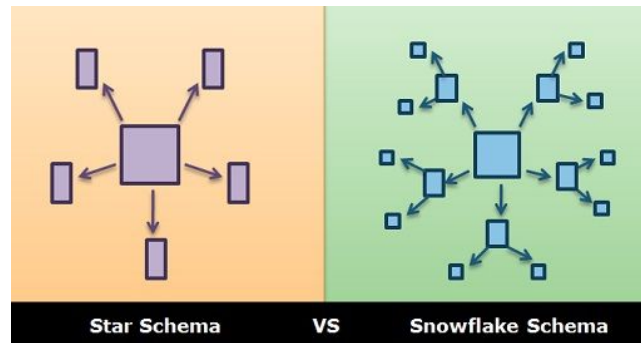
Progettato per dati con relazioni molto complesse (es social network, sistemi di raccomandazione)

- Strutturato tramite **Vertici** (nodi/entità) e **Archi** (relazioni direzionate)
- Superiore per gli attraversamenti di percorsi dinamici e relazioni molti-a-molti imprevedibili, che nei database relazionali richiederebbero query ricorsive lentissime
- Utilizza linguaggi dichiarativi orientati ai pattern, come **Cypher** per Neo4j

2.4 Schemi Analitici (Data Warehouse)

I database analitici denormalizzati utilizzano convenzioni strutturali specifiche :

- **Star schema**: al centro si trova una *tabella dei fatti* (che registra gli eventi), circondata a raggiera da *tabelle delle dimensioni* (il chi, cosa, dove, quando)
- **Snowflake schema**: una variante in cui le tabelle delle dimensioni sono ulteriormente divise in sotto-dimensioni



2.5 Altri Modelli (NoSQL)

- **Key-Value Stores:** i NoSQL più semplici Usano una mappa (hash table) per accedere a valori arbitrari unicamente tramite una chiave (es Redis, DynamoDB)
- **Wide-Column Stores:** pensati per database enormi (svariati GB distribuiti) Funzionano come mappe sparse distribuite identificate da (row-id, column-family, column-id, timestamp) (es Cassandra)
- **Vector Stores:** memorizzano **embeddings** (vettori multi-dimensionali generati da modelli ML) Permettono **ricerche per similarità** e supportano architetture di intelligenza artificiale come la Retrieval-Augmented Generation (RAG)

3 Replication

La replica consiste nel mantenere una copia degli stessi dati su più macchine (nodi). Viene utilizzata per aumentare la **scalabilità** (distribuendo il carico di lettura/scrittura), garantire la **tolleranza ai guasti** (alta disponibilità in caso di crash) e ridurre la **latenza** geografica avvicinando i dati agli utenti.

3.1 Single-Leader Replication (Active/Passive)

È l'approccio più comune nei database relazionali e non.

- **Leader:** L'unico nodo autorizzato ad accettare le operazioni di scrittura.
- **Followers:** Nodi in sola lettura che replicano il log delle modifiche inviato dal leader. Aggiungere follower permette di scalare le letture (read-scaling).
- **Sincrona vs Asincrona:** Nella replica sincrona il leader attende la conferma del follower (maggiore durabilità, minore performance). Nell'asincrona non attende (molto veloce, ma si rischia la perdita di dati se il leader fallisce in modo permanente). In genere si usa un approccio *semi-sincrono*.

3.2 Handling Node Outages

Per garantire l'alta disponibilità, i sistemi devono gestire in modo sicuro i guasti dei singoli nodi:

- **Follower Failure (Catch-up Recovery):** Se un follower si riavvia o si riprende da un guasto, legge dal proprio log locale l'ultima transazione elaborata. Dopodiché, si connette al leader e richiede tutte le modifiche successive per "mettersi in pari".
- **Leader Failure (Failover):** Se il leader cade (evento solitamente rilevato tramite un timeout), il sistema deve eleggere un nuovo leader (spesso il follower più aggiornato per minimizzare la perdita di dati) e riconfigurare i client affinché inviino le scritture al nuovo nodo. È fondamentale gestire il rischio di **split-brain**, che si verifica se il vecchio leader torna online credendo di essere ancora in carica.

3.3 Replication Lag e Consistenza

Il ritardo di replica tra leader e follower causa temporanee incongruenze ("eventual consistency"). Per mitigarle, i sistemi offrono modelli di consistenza specifici:

- **Reading Your Own Writes:** Garantisce che un utente veda sempre i dati o le modifiche che ha appena inserito (es. leggendo dal leader per i propri dati).
- **Monotonic Reads:** Assicura che un utente non veda il tempo scorrere "all'indietro" a causa di letture fatte su repliche aggiornate in tempi diversi.
- **Consistent Prefix:** Garantisce che chiunque legga una sequenza causale di scritture la veda nello stesso e corretto ordine logico.

3.4 Multi-Leader Replication

Più nodi (leader) accettano scritture in modo indipendente, inoltrandosi reciprocamente le modifiche.

- Ideale per architetture geo-distribuite in più datacenter o per app collaborative che funzionano anche offline (come i sync engine, es. Google Docs).
- Il problema principale sono i **conflitti di scrittura** causati da modifiche concorrenti allo stesso record.
- **Risoluzione conflitti**: Esistono diversi approcci per gestirli:
 - *Conflict Avoidance*: Instradare tutte le scritture relative a uno specifico record o utente sempre allo stesso leader (leader affinity), evitando i conflitti alla radice.
 - *Last Write Wins (LWW)*: Ogni scrittura riceve un timestamp; in caso di conflitto, il sistema conserva solo il valore con il timestamp più recente. Questo approccio è semplice ma causa **perdita di dati** scartando silenziosamente le altre scritture concorrenti.
 - *Manual / Deferred Resolution*: Il database conserva tutte le versioni in conflitto (chiamate *siblings*) e delega la risoluzione all'applicazione o chiede all'utente di scegliere (es. in CouchDB).
 - *Automatic Resolution*: Algoritmi che fondono i dati in automatico in base al tipo di dato. Esempi principali sono **OT (Operational Transformation)**, usato negli editor di testo collaborativi che trasforma gli indici dei caratteri inseriti, e **CRDTs (Conflict-free Replicated Datatypes)**, che assegnano ID immutabili agli elementi (usato in Riak, Redis).

3.5 Leaderless Replication

Non esiste un leader: qualsiasi replica può accettare scritture direttamente dai client.

- Si basa sul concetto di **Quorum per letture e scritture**: interrogando/scrivendo in parallelo su più nodi, se i nodi che confermano la scrittura (w) sommati a quelli interrogati in lettura (r) superano il numero totale dei nodi (n), ovvero $w + r > n$, si ha la garanzia matematica che la lettura intercetti almeno un nodo con il dato più recente.
- Per identificare la versione corretta dei dati e individuare le scritture concorrenti si usano version numbers o "version vectors".
- Meccanismi di allineamento: le repliche si aggiornano tra loro tramite **Read Repair** (correzione triggerata in fase di lettura di un dato obsoleto), **Hinted Handoff** (salvataggio temporaneo di messaggi per un nodo offline) e **Anti-Entropy** (processi di background che allineano i dati).

3.6 Detecting Concurrent Writes

Per poter risolvere i conflitti in modo esplicito (sia in multi-leader che leaderless), il database deve prima di tutto essere in grado di rilevare quali scritture sono effettivamente *concorrenti* (ossia avvenute nello stesso momento senza che l'una sapesse dell'altra).

- Il server mantiene un **version number** per ogni chiave e lo incrementa a ogni scrittura.

- Quando un client legge un dato, riceve il valore corrente e il suo version number. Per poter aggiornare quel dato, il client **deve includere il version number** precedentemente letto.
- Ricevendo la scrittura, il database sa che questa sovrascrive tutte le versioni precedenti o uguali a quel numero, ma deve mantenere le versioni con un version number superiore, considerandole **scritture concorrenti** (che diventano *siblings* da risolvere).
- In sistemi con repliche multiple, si utilizza un **Version Vector**: un array che contiene un version number separato per ogni singola replica. Confrontando matematicamente i Version Vectors (es. V_a e V_b), il sistema può determinare rigorosamente se le versioni sono identiche, se una è successiva all'altra, o se sono state generate in modo del tutto concorrente.

4 Sharding

Lo sharding (o partizionamento) consiste nel suddividere un database di grandi dimensioni in sottoinsiemi più piccoli e indipendenti (chiamati shard o partizioni), assegnando ciascun sottoinsieme a una macchina (nodo) diversa. L'obiettivo principale è garantire la **scalabilità orizzontale**, distribuendo sia i volumi di dati che il carico delle operazioni (letture e scritture), ed è spesso combinato con tecniche di replica.

4.1 Strategie di Partizionamento

- **Sharding by Key Range:** I record vengono assegnati in base a un intervallo contiguo di chiavi (es. dalla "A" alla "D"). È un approccio ottimale per le *range queries* (interrogazioni su intervalli), ma espone al rischio di **hotspot** (sovraccarichi) se l'accesso ai dati si concentra in modo asimmetrico su chiavi adiacenti.
- **Sharding by Hash of Key:** Si applica una funzione hash alla chiave per distribuire i dati in modo pseudo-casuale e uniforme, bilanciando perfettamente il carico. Questo rende le *range queries* inefficienti. Spesso si crea un numero fisso e molto elevato di shard logici, distribuendone poi diversi per ogni nodo fisico.
- **Sharding by Hash Range:** Usa un numero dinamico di shard assegnando a ciascuno un intervallo di valori hash, evitando di dover indovinare il numero giusto di shard a priori.
- **Sharding by Consistent Hashing:** Le chiavi vengono mappate su un "anello" di hash e i nodi si spartiscono vari segmenti di questo anello. Riduce al minimo i trasferimenti di dati: aggiungendo o rimuovendo un nodo, solo una piccola frazione di chiavi (circa $1/N$) si sposta.

4.2 Workload Sbilanciati e Hot Spot

Anche usando l'hashing, l'accesso estremo a un'unica chiave (es. un post di una celebrità sui social) causerà l'invio di tutto il traffico a un solo nodo. Si può mitigare il problema a livello applicativo tramite il **key salting**, aggiungendo numeri casuali alla fine della chiave per forzare la distribuzione del carico su più nodi, pagando però il prezzo di letture più complesse.

4.3 Rebalancing (Ribilanciamento)

Processo costoso tramite il quale si spostano dati da uno shard all'altro (es. per l'aggiunta di nuovi nodi o crescita del database). Può essere:

- **Automatico:** reattivo ma rischioso, potrebbe causare fallimenti a cascata se innescato da rallentamenti di rete transitori.
- **Manuale:** genera suggerimenti automatici, ma richiede l'approvazione umana per evitare sorprese operative.

4.4 Request Routing (Service Discovery)

Per instradare la query di un client al nodo che detiene effettivamente lo shard richiesto, l'architettura può prevedere che il client contatti a caso un nodo (che poi farà da passacarte), oppure usare un **routing tier** (una sorta di load-balancer intelligente). Per tenere traccia in tempo reale della mappatura "shard → nodo", i sistemi si affidano spesso a servizi di coordinamento come **ZooKeeper** (usato ad es. da HBase) o usano il protocollo di consenso Raft.

4.5 Secondary Indexes nello Sharding

Gli indici secondari permettono ricerche non basate sulla chiave primaria, ma si complicano con lo sharding. Due approcci:

- **Local Secondary Index (Document-partitioned):** Ogni nodo indicizza unicamente i record presenti nel proprio shard. Le **scritture sono veloci** e locali, ma le letture sono lente e inefficienti perché richiedono il paradigma "scatter/gather": il database deve interrogare tutti gli shard in parallelo e unire i risultati, diventando vulnerabile alla "tail latency" dello shard più lento.
- **Global Secondary Index (Term-partitioned):** Esiste un unico indice globale, che viene a sua volta shardato in base alla chiave dell'indice stesso. Le **letture sono molto veloci** (vanno dirette al singolo shard che ha l'indice), ma le scritture sono lente e complesse perché l'aggiornamento di un record può coinvolgere l'aggiornamento di un indice che risiede in un nodo totalmente diverso, richiedendo coordinamento.

5 Consistency (Consistenza e Consenso)

Nei database relazionali tradizionali su singola macchina, la consistenza fa parte delle proprietà ACID e indica la preservazione dei vincoli di integrità dei dati. Invece, nei sistemi distribuiti e replicati, il problema della consistenza si sposta su una domanda cruciale: *quali valori possono restituire le letture a fronte di modifiche concorrenti e guasti?*.

5.1 Eventual Consistency vs Strong Consistency

Per gestire le repliche, esistono due filosofie architetturali contrapposte:

- **Eventual Consistency (Consistenza Finale):** Assicura che, se non vi sono nuove scritture, tutte le repliche prima o poi convergeranno allo stesso stato. Delega allo sviluppatore la gestione di dati temporaneamente obsoleti e la risoluzione dei conflitti, ma massimizza la tolleranza ai guasti.
- **Strong Consistency (Consistenza Forte):** Il sistema maschererà completamente la replicazione, comportandosi come se ci fosse una singola e unica copia dei dati. Semplifica enormemente il lavoro degli sviluppatori, ma ha un costo elevato in termini di performance.

5.2 Linearizability (Consistenza Forte)

La *Linearizability* (o consistenza atomica) è la definizione formale più comune di Strong Consistency.

- Offre una garanzia di **recency** (aggiornamento): una volta che una scrittura è stata confermata, qualsiasi lettura successiva (da parte di chiunque) deve restituire il nuovo valore appena scritto o uno ancora più recente, impedendo di "tornare indietro nel tempo".
- Il limite principale della linearizzabilità è la **lentezza**: per garantirla, il tempo di risposta per letture e scritture deve essere almeno proporzionale all'incertezza e ai ritardi imprevedibili della rete.

5.3 Il Teorema CAP

Un noto teorema che descrive i trade-off della consistenza distribuita. Afferma che, qualora si verifichi una **Partizione di Rete (P)** (ovvero un'interruzione della comunicazione tra i nodi), il sistema non può garantire contemporaneamente **Linearizzabilità (C)** e **Disponibilità (A)**.

- **Sistemi CP:** Preferiscono la consistenza forte. Se la rete si divide, bloccano o rifiutano le richieste fino alla risoluzione del problema.
- **Sistemi AP:** Preferiscono la disponibilità. In caso di partizione, continuano a rispondere alle richieste pur rischiando di restituire dati obsoleti o in conflitto.

5.4 Algoritmi di Consenso

Per ottenere la Linearizzabilità e gestire in sicurezza il failover automatico dei leader (evitando catastrofi come lo *split-brain*), i sistemi moderni (es. etcd, ZooKeeper, Spanner, CockroachDB) ricorrono a protocolli di consenso fault-tolerant come **Raft**, **Paxos**, **VSR** o **Zab**.

- Il consenso permette a macchine distribuite di **trovare un accordo** sull'elezione di un leader o sull'ordine logico delle operazioni in un Log Condiviso (Shared Log).

- Si basa su meccanismi di **Quorum** (una maggioranza stretta, es. servono 3 nodi per tollerare 1 guasto, o 5 per tollerarne 2). La leadership e il commit dipendono dall'intersezione matematica dei quorum, non dall'unanimità.

5.5 Serializzabilità vs Linearizzabilità

Le due nozioni spesso si confondono, ma operano in ambiti diversi. Se vengono combinate formano la cosiddetta **Strict Serializability**.

- **Serializzabilità**: È una garanzia di *isolamento transazionale* (ACID). Assicura che le transazioni concorrenti si comportino come se fossero eseguite in serie, ma non impone l'ordine del tempo reale in cui sono avvenute (manca di *recency*).
- **Linearizzabilità**: È una garanzia di *recency* sui singoli oggetti. Le operazioni appaiono istantanee e rispettano rigorosamente l'ordine del tempo reale, ma non raggruppa operazioni multiple in transazioni.
- **Strict Serializability**: Unisce entrambe le garanzie. Le transazioni appaiono eseguite in un ordine seriale che rispetta anche le precedenze del tempo reale (usata in sistemi come Spanner e FoundationDB).

5.6 Distributed Transactions e Two-Phase Commit (2PC)

Nelle architetture distribuite (es. con sharding), una singola transazione potrebbe dover scrivere su più nodi contemporaneamente. Per garantire la proprietà di **Atomicità** dell'intera transazione su scala distribuita (tutti i nodi fanno il commit, o nessuno lo fa), si utilizza il protocollo **Two-Phase Commit (2PC)** coordinato da un nodo centrale. Il protocollo opera in due fasi:

- **Fase 1 (Prepare)**: Il coordinatore invia una richiesta a tutti i nodi partecipanti chiedendo se sono pronti al commit. Ogni nodo controlla i vincoli di integrità e salva i dati sul proprio disco in modo permanente. Rispondendo "sì", il partecipante rinuncia irrevocabilmente al diritto di fare abort volontario.
- **Fase 2 (Commit/Abort)**: Se tutti i partecipanti votano "sì", il coordinatore prende la decisione definitiva, la salva sul proprio disco (*commit point*), e istruisce tutti i nodi a procedere con il commit effettivo. Se anche un solo nodo risponde "no" o va in timeout, il coordinatore comunica a tutti di effettuare un abort.
- **Il problema del blocco (Coordinator Failure)**: Il difetto principale del 2PC è il rischio di blocchi. Se il coordinatore va in crash o la rete si disconnette *dopo* che i partecipanti hanno votato "sì" ma prima che abbiano ricevuto la direttiva finale di commit, i partecipanti restano bloccati in un limbo. Non possono né fare commit né abortire, e sono costretti a mantenere i *lock* sulle righe modificate, impedendo l'accesso ad altre transazioni concorrenti fino al ripristino del coordinatore.

6 Batch Processing

Il batch processing (elaborazione batch) consiste nell'elaborazione offline di grandi volumi di dati precedentemente raccolti e archiviati. A differenza dei sistemi online dove la priorità è un basso tempo di risposta, nei sistemi batch la metrica principale è il **throughput**, ovvero la quantità di dati elaborata per unità di tempo.

6.1 Caratteristiche Principali

- **Flusso tipico:** 1) Lettura di un enorme dataset di input, 2) Esecuzione di una computazione sull'intero dataset, 3) Scrittura di un grande dataset di output.
- **Tolleranza ai guasti:** Nei sistemi batch, gestire i fallimenti è relativamente semplice. Dato che l'input è immutabile e il job rigenera l'output da zero, se un task fallisce basta scartare l'output parziale e ritentare l'esecuzione su un'altra macchina.

6.2 L'Analogia con Unix e i Sistemi Distribuiti

Per comprendere i framework batch distribuiti, si può pensare alle pipeline dei comandi Unix (es. `cat | awk | sort | uniq`) che leggono, trasformano, ordinano e filtrano dati. In scala distribuita:

- I **Distributed Filesystems** (es. HDFS) o gli **Object Stores** (es. Amazon S3) sostituiscono il file system locale, spezzando file giganteschi in grandi blocchi replicati su più macchine.
- I **Job Orchestrator** (es. YARN, Kubernetes) sostituiscono il sistema operativo, allocando le risorse (CPU, RAM) e programmando i task sui vari nodi del cluster.

6.3 Core Orchestrator Components

Per gestire e programmare l'esecuzione dei job all'interno di un cluster, i framework di orchestrazione si affidano a componenti standardizzati:

- **Task Executors:** Demoni in esecuzione su ciascun nodo del cluster (es. NodeManager in YARN, kubelet in Kubernetes). Sono incaricati di avviare i task, recuperare il codice eseguibile, monitorarne l'esecuzione e tracciarne lo stato e l'uso delle risorse. Garantiscono inoltre l'isolamento delle prestazioni e della sicurezza, spesso tramite meccanismi a livello di OS come i *cgroups* di Linux.
- **Resource Manager:** Mantiene una visione centralizzata dello stato globale del cluster, tracciando le risorse disponibili (CPU, GPU, RAM, dischi) e la salute dei nodi. Poiché la centralizzazione può creare colli di bottiglia, i Resource Manager si appoggiano tipicamente a sistemi di coordinamento esterni per memorizzare lo stato del cluster (es. ZooKeeper per YARN, etcd per Kubernetes).
- **Scheduler:** Gestisce le richieste di avvio, stop o interrogazione dei job. Sulla base delle informazioni fornite dal Resource Manager e dei requisiti del job, prende le decisioni di posizionamento dei task (data locality) e istruisce gli Executor per avviarli.

6.4 Modello MapReduce

Introdotta da Google nel 2004 e resa celebre da Apache Hadoop, è un modello di programmazione che divide il job in due funzioni principali scritte dall'utente:

- **Map:** Una funzione *stateless* chiamata per ogni singolo record di input, che estrae ed emette coppie (chiave, valore).
- **Shuffle & Sort:** Una fase implicita in cui il framework raggruppa e ordina automaticamente tutte le coppie prodotte in base alla chiave.
- **Reduce:** Riceve una chiave con la lista associata di tutti i suoi valori e li aggrega (es. sommandoli o contandoli) per produrre l'output finale.
- **Limiti:** Può risultare lento perché forza la scrittura su disco dei risultati intermedi tra i vari job, e rende faticosa l'implementazione da zero di operazioni complesse come i JOIN.

6.5 Modello Dataflow (Apache Spark, Flink)

Nato per superare i limiti di MapReduce, questo modello (utilizzato da Spark e Flink) rappresenta il job come un **Grafo Diretto Aciclico (DAG)** di operazioni.

- Offre nativamente **operatori di alto livello** come Join, GroupByKey e Filter, senza limitarsi alle sole fasi di Map e Reduce.
- **Ottimizzazioni di performance:** Evita di ordinare i dati se non è strettamente necessario, *fonde* (fuse) operazioni compatibili (es. map e filter) in un singolo task e processa i dati a cascata (pipeline), riducendo le letture/scritture su disco.
- **Fault tolerance in-memory:** Sistemi come Spark tengono i dati intermedi in RAM; se un dato viene perso a causa di un guasto, il sistema tiene traccia delle trasformazioni (lineage) e si limita a ricalcolare solo la porzione mancante a partire dall'input originale.

7 Stream Processing

Lo stream processing consiste nell'elaborazione continua di dati non limitati (unbounded) che arrivano progressivamente nel tempo, a differenza del batch processing che opera su dataset statici. L'obiettivo è minimizzare la latenza ed elaborare i dati quasi in tempo reale.

7.1 Eventi e Message Brokers

I dati nei sistemi stream sono rappresentati come **eventi**: piccoli oggetti immutabili e auto-contenuti che descrivono qualcosa accaduto in un preciso istante (spesso dotati di timestamp). Per trasmettere questi eventi in modo affidabile tra producer e consumer, si utilizzano i **Message Brokers**:

- **Broker Tradizionali (es. AMQP/JMS)**: I messaggi vengono eliminati dalla coda non appena il consumer ne conferma l'elaborazione tramite un **acknowledgment**. Il carico viene bilanciato per singolo messaggio, ma se l'acknowledgment non arriva (es. guasto del consumer), il broker ritenta l'invio tramite la **redelivery**. La combinazione di redelivery e load balancing può causare la **perdita dell'ordine causale** degli eventi (un messaggio ritardato potrebbe essere elaborato dopo uno più recente). Inoltre, per evitare che messaggi mal-formati causino loop infiniti di retry, i broker li isolano in apposite code chiamate **Dead Letter Queues (DLQs)**.
- **Broker Log-Based (es. Apache Kafka)**: I messaggi vengono scritti su disco in un log *append-only* suddiviso in **partizioni**. I consumer tracciano il loro avanzamento tramite un **offset** e i messaggi non vengono cancellati subito. Questo garantisce l'ordine all'interno della singola partizione e permette il **replay** degli eventi passati (es. riprocessare l'ultimo giorno di dati resettando l'offset).

7.2 Change Data Capture (CDC) e Log Compaction

Il CDC è una tecnica che osserva tutte le modifiche scritte in un database e le estrae sotto forma di stream di eventi.

- Risolve il problema delle **dual writes** (scrivere contemporaneamente sul DB principale e su sistemi derivati come cache o indici di ricerca), che espongono a race conditions e inconsistenze.
- Con il CDC, il database principale funge da System of Record e i sistemi derivati consumano asincronamente lo stream dei cambiamenti per aggiornarsi nello stesso identico ordine.
- Per evitare che i log crescano all'infinito, si usa la **Log Compaction**: il sistema scarta i valori più vecchi sovrascritti e mantiene solo l'aggiornamento più recente per ogni chiave. Questo fa sì che lo spazio su disco dipenda dallo stato corrente e non dall'intera storia delle scritture, facilitando la ricostruzione da zero dei sistemi derivati.

7.3 Command Query Responsibility Segregation (CQRS)

Il pattern CQRS prevede la separazione tra la fase di scrittura e la fase di lettura.

- I dati vengono salvati in un event log *write-optimized* e successivamente tradotti in rappresentazioni multiple (viste) *read-optimized*.

- Questo approccio permette di mantenere i dati normalizzati in fase di scrittura e de-normalizzati nelle viste di lettura, alimentando applicazioni diverse a partire da un unico stream.
- Il limite principale è il **read-your-own-write lag**: un utente potrebbe effettuare una modifica e interrogare subito dopo la vista di lettura, trovandola temporaneamente obsoleta.

7.4 Casi d'Uso e Join negli Stream

Oltre al monitoraggio e agli alert (es. rilevamento frodi finanziarie tramite **Complex Event Processing - CEP**), lo stream processing permette di mantenere viste materializzate e calcolare statistiche su finestre temporali (**Stream Analytics**). I **Join** negli stream sono particolarmente complessi perché i dati sono infiniti:

- **Stream-Stream Joins (Window Joins)**: Correlano eventi provenienti da due stream diversi (es. un utente fa una ricerca e poi clicca su un risultato). Poiché il tempo tra i due eventi è imprevedibile, si definisce una **finestra temporale** (es. un'ora) entro cui il join è valido, mantenendo lo stato in memoria.
- **Stream-Table Joins (Enrichment)**: Arricchiscono un evento in tempo reale con dati anagrafici (es. id utente → età, posizione). Per evitare di interrogare un DB remoto per ogni evento (causando colli di bottiglia), il framework mantiene una **copia locale** della tabella, tenendola aggiornata tramite un flusso CDC.
- **Table-Table Joins (Materialized Views)**: Tratta entrambe le tabelle di input come changelogs. Applica ogni singolo cambiamento in modo incrementale al risultato del join, mantenendolo costantemente aggiornato come se fosse una vista materializzata.

7.5 Tolleranza ai Guasti (Fault Tolerance)

In un sistema continuo, non è possibile scartare l'output e ricominciare da capo in caso di crash come nel batch processing. Si usano due approcci principali per recuperare lo stato:

- **Microbatching** (es. Spark Streaming): Suddivide lo stream in blocchi piccolissimi (es. 1 secondo) elaborati come mini-job batch. Se un task fallisce, si scarta il singolo microbatch e lo si ritenta.
- **Checkpointing** (es. Flink): Genera periodicamente degli snapshot dello stato corrente degli operatori e li salva su storage durevole. In caso di crash, il sistema riavvia l'operatore riprendendo l'esecuzione a partire dall'ultimo checkpoint salvato, scartando i risultati parziali.

Per garantire garanzie di semantica **Exactly-Once** anche nei confronti di sistemi esterni (evitando aggiornamenti doppi dovuti ai riavvii), il framework deve implementare protocolli di **Atomic Commit** o modellare le operazioni affinché siano strettamente **idempotenti**.

8 MongoDB: Architettura e Query

MongoDB è un database orientato ai documenti, caratterizzato da uno schema flessibile (schema-less o schema-flexible) in cui i dati sono memorizzati come documenti JSON-like (più precisamente in formato binario BSON). È progettato per garantire alta disponibilità e scalabilità orizzontale attraverso l'uso combinato di replica e sharding.

8.1 Replication (Replica Sets)

In MongoDB, la replicazione è gestita tramite i **Replica Sets**, che implementano un'architettura di tipo Single-Leader.

- **PRIMARY (Leader):** È l'unico nodo autorizzato ad accettare le operazioni di scrittura, occupandosi di coordinarle e ordinarle.
- **SECONDARY (Followers):** Repliche in sola lettura che si aggiornano in modo asincrono a partire dai log del primario. I client possono leggere dai secondari impostando una specifica "read preference" (accettando una consistenza finale).
- **Elezioni e Failover:** In caso di caduta del nodo primario, i nodi secondari indicano un'elezione basata su consenso per nominare un nuovo primario. Per evitare situazioni di stallo o *split-brain* in caso di partizioni di rete, è best practice utilizzare un numero dispari di membri votanti (es 3 o 5 nodi), assicurando così il raggiungimento di una maggioranza. Se non c'è una maggioranza disponibile, il cluster smette di accettare scritture.

8.2 Sharding (Scalabilità Orizzontale)

Lo sharding permette di dividere database molto grandi e carichi di lavoro elevati su più macchine. Un cluster sharded in MongoDB è composto da tre elementi:

- **Config Servers:** Un replica set (tipicamente di 3 nodi) che memorizza i metadati del cluster e mappa quali blocchi di dati (chunk) appartengono a quale shard.
- **Shard Servers:** Nodi che contengono effettivamente i sottoinsiemi dei dati partizionati. In produzione, ogni singolo shard è implementato a sua volta come un Replica Set per garantire la tolleranza ai guasti.
- **Mongos (Query Router):** Agisce da entry-point ("passacarte") per i client. Interroga i Config Servers per scoprire dove risiedono i dati e instrada le letture e le scritture verso lo shard corretto.

MongoDB supporta due principali strategie di sharding basate sulla *shard key*: lo sharding basato su intervalli (**Range-based sharding**) e quello basato sull'hash della chiave (**Hashed sharding**).

8.3 Tipi di Query in MongoDB

MongoDB non usa SQL, ma il **MongoDB Query Language (MQL)**, basato su JavaScript. Le operazioni si dividono in query di base (CRUD) e pipeline di aggregazione.

8.3.1 Query di Lettura (find)

Il comando base per la lettura è `dbcollectionfind()` Supporta varie funzionalità:

- **Ricerca per `_id`:** Ricerca esatta sul campo identificativo (tipicamente un `ObjectId`)
- **Filtri con Operatori Condizionali:** Uso di operatori come `$lt` (minore di), `$gt` (maggiore di), `$in` o l'operatore logico `$or` per combinare più criteri. È supportato anche l'utilizzo nativo di **Espressioni Regolari (Regex)**
- **Query su Array:** Permettono di filtrare documenti in cui un array contiene un valore specifico, o verificare combinazioni con `$all` (contiene tutti i valori specificati) o `$nin` (non contiene i valori)
- **Dot Notation (Oggetti Annidati):** Permette di interrogare in profondità attributi all'interno di sotto-documenti (es `'mayorparty' : 'X'`), senza dover ricorrere a complessi JOIN come nei database relazionali
- **Proiezioni:** Permettono di limitare il payload di ritorno selezionando solo i campi necessari (es `name: 1, _id: 0`)

8.3.2 Pipeline di Aggregazione (aggregate)

Per raggruppamenti, calcoli statistici o trasformazioni complesse, si utilizza `dbcollectionaggregate()`, che funziona a "stadi" sequenziali (Pipeline Model) simili ai comandi in pipe di Unix. I principali operatori sono:

- **`$match`:** Filtra i documenti che passeranno allo stadio successivo (equivalente a `WHERE` in SQL)
- **`$group`:** Raggruppa i documenti per una specifica chiave e calcola metriche aggregate tramite operatori come `$sum`, `$avg`, `$first`, `$push`
- **`$sort`:** Ordina i risultati
- **`$project`:** Modella o estrae attributi per formare nuovi documenti di output
- **`$unwind`:** "Appiattisce" un array, srotolandolo e generando un documento separato per ogni elemento contenuto nell'array
- **`$lookup`:** Esegue una vera e propria "left outer join" incrociando i dati del documento corrente con un'altra collezione
- **`$setWindowFields`:** Operatore introdotto per eseguire funzioni analitiche e finestre logiche (es somme progressive o `rank`), equivalente alla clausola `PARTITION BY` di SQL
- **`$merge`:** Salva il risultato finale dell'aggregazione in una collezione fisica

8.3.3 Query di Aggiornamento e Cancellazione

- Le operazioni `updateOne`, `updateMany`, `deleteOne` e `deleteMany` alterano i dati
- Durante un `update`, per modificare solo campi specifici senza sovrascrivere l'intero documento si utilizza l'operatore **`$set`**
- L'operatore **`$inc`** permette di incrementare numericamente un campo in modo completamente atomico

8.4 Funzionalità Avanzate

- **Schema Validation (\$jsonSchema):** Sebbene nasca schemaless, MongoDB permette di forzare regole sui dati in ingresso (obbligatorietà di campi, tipi di dato come stringhe o interi) tramite uno schema di validazione a livello di database
- **Transazioni Multi-Documento:** MongoDB supporta le transazioni ACID per eseguire in blocco modifiche multiple su diverse collezioni, ma solo se l'architettura è configurata come Replica Set o Cluster Sharded
- **Change Streams:** Una funzione di stream processing che si aggancia al log di replicazione, permettendo ad un'applicazione esterna di "ascoltare" in tempo reale (come se fosse un pattern CDC) qualsiasi operazione di inserimento, aggiornamento o cancellazione (es tramite `dbcollectionwatch()`)

9 Neo4j e Database a Grafo

Neo4j è un database a grafo nativo, progettato per eccellere in scenari in cui le connessioni tra i dati sono importanti tanto quanto i dati stessi (es motori di raccomandazione, rilevamento frodi, analisi di reti)

9.1 Property Graph Model (Modello a Grafo di Proprietà)

Il modello di memorizzazione di Neo4j si basa su tre concetti fondamentali:

- **Nodi (Vertici):** Rappresentano le entità (es persone, aziende) Possono avere zero o più **Labels** (etichette) che li categorizzano (es :Person, :Movie)
- **Relazioni (Archi):** Sono le connessioni **direzionate** tra due nodi Devono avere sempre esattamente un **Type** (tipo, convenzionalmente scritto in maiuscolo, es :ACTED_IN)
- **Proprietà:** Attributi memorizzati sotto forma di coppie chiave-valore Possono essere salvati **sia sui nodi che sulle relazioni** (es il ruolo di un attore in un film è una proprietà della relazione)

9.2 Cypher Query Language

È un linguaggio dichiarativo progettato specificamente per interrogare i grafi tramite **pattern matching** visivo (stile "ASCII-art")

- Esprime i percorsi in modo molto intuitivo: (p:Person)-[r:ACTED_IN]->(m:Movie)
- Permette **attraversamenti a lunghezza variabile** (es percorsi da 1 a 4 salti: -[*14]->) in modo nativo ed efficiente, superando i forti limiti prestazionali delle *Recursive CTE* del mondo SQL
- Permette il calcolo automatico degli **Shortest Path** (percorsi più brevi) tra due nodi

9.3 Architettura e Transazioni

- **ACID:** A differenza di molti NoSQL, Neo4j garantisce transazioni ACID complete (tramite BEGIN, COMMIT, ROLLBACK) e Strong Consistency, un requisito essenziale per preservare l'integrità referenziale e la topologia del grafo
- **Index-Free Adjacency:** I nodi mantengono veri e propri puntatori fisici in memoria verso i nodi adiacenti Questo rende i "join" (attraversamenti) velocissimi indipendentemente dalla dimensione totale del database, superando la necessità di utilizzare indici globali per esplorare le relazioni
- **Clustering:** Usa un'architettura basata sul consenso **Raft** per replicare i dati tra i nodi primari, garantendo alta disponibilità Le scritture vanno al leader, mentre le letture possono scalare sui nodi replicati

9.4 Modellazione e Schema

- Regola euristica di design: i *nomi* dei casi d'uso diventano le **Label** dei nodi, i *verbi* diventano i **Type** delle relazioni, i *dettagli* diventano le **Proprietà**
- **Graph Types:** Sebbene il database nasca come *schema-free*, le versioni recenti supportano un "open schema" che permette di forzare tipi di dato, proprietà obbligatorie e chiavi uniche (es vincolare un nodo City ad avere un `name` di tipo stringa)

9.5 Il Pattern "Thin Node" (Polyglot Persistence)

Nell'ambito di applicazioni Data-Intensive, gli algoritmi sui grafi sono fortemente legati alla memoria RAM (*memory-bound*) Per evitare di saturare la *page cache* del database:

- Si applica il pattern dei **Thin Node** (nodi leggeri): in Neo4j si memorizzano solo gli identificativi univoci (*_id*) e pochi flag essenziali per le query topologiche
- Tutti i dati "pesanti" (testi lunghi, descrizioni, anagrafiche complete) vengono delegati a un DBMS documentale come MongoDB Questa orchestrazione ibrida permette di estrarre percorsi e cicli complessi su Neo4j a velocità estreme, recuperando poi i payload testuali su Mongo (*Application-Level Join*)

10 Amazon DynamoDB

DynamoDB è un database NoSQL serverless e completamente gestito offerto da Amazon Web Services (AWS). Supporta sia il modello chiave-valore che quello a documenti. È progettato per offrire latenze basse e prevedibili su larghissima scala, replicando automaticamente i dati su SSD attraverso molteplici "Availability Zones" all'interno delle regioni AWS, sgravando l'utente dalla gestione dei nodi server.

10.1 Modello dei Dati e Chiavi

A differenza dei database relazionali, in DynamoDB non esiste uno schema predefinito per gli attributi, ad eccezione della chiave primaria. L'architettura logica si basa su **Tabelle** che contengono **Item** (elementi), a loro volta composti da **Attributi**. La scelta della chiave primaria è fondamentale e può essere di due tipi:

- **Simple Primary Key (solo Partition key):** Un solo attributo identifica univocamente l'item.
- **Composite Primary Key (Partition key + Sort key):** Permette di raggruppare più item sotto la stessa partition key, mantenendoli fisicamente ordinati in base alla sort key.

10.2 Partizionamento e Problema degli Hot Spot

DynamoDB distribuisce i dati su partizioni fisiche calcolando l'hash della partition key. È cruciale scegliere una partition key con alta cardinalità per distribuire uniformemente le letture e le scritture. Se troppe richieste colpiscono la stessa chiave (bassa cardinalità), si crea una **Hot Partition**, causando rallentamenti ("throttling") e riducendo le prestazioni generali del database.

10.3 Letture e Consistenza

Mantenendo copie multiple dei dati, DynamoDB offre due modelli di consistenza in lettura:

- **Eventually Consistent Read (default):** L'operazione potrebbe restituire dati leggermente obsoleti se effettuata subito dopo una scrittura, ma ha un costo inferiore.
- **Strongly Consistent Read:** Interroga il leader per restituire il dato più aggiornato, ma ha un costo più elevato.

10.4 Indici Secondari

Poiché DynamoDB permette ricerche efficienti solo tramite la chiave primaria, per supportare pattern di accesso alternativi è necessario creare degli indici:

- **Global Secondary Index (GSI):** Crea un indice con una partition key e una sort key totalmente differenti da quelle della tabella base.
- **Local Secondary Index (LSI):** Mantiene la stessa partition key della tabella principale, ma utilizza una sort key differente.

10.5 Query, Scan e Limiti Architetture

- **Query:** È l'operazione efficiente di base Richiede tassativamente una condizione di uguaglianza sulla partition key, a cui si possono aggiungere condizioni logiche sulla sort key
- **Scan:** Legge l'intera tabella o indice per filtrare i risultati a posteriori È considerato un **anti-pattern in produzione** per tabelle di grandi dimensioni perché consuma enormi capacità di lettura ed è lento e costoso

A differenza dei DBMS relazionali, DynamoDB **non supporta** JOIN tra tabelle, aggregazioni OLAP o query ad-hoc arbitrarie L'intero design del database deve partire dai pattern di accesso (le query) che l'applicazione dovrà fare

10.6 Global Tables e Gestione della Capacità

Per le architetture multi-regione, DynamoDB offre le **Global Tables**, repliche multi-leader (multi-active) in cui ogni regione può accettare scritture I conflitti vengono risolti in automatico tramite la regola del *Last Write Wins* Infine, il sistema dei costi e delle performance (Throughput) si basa su due modalità:

- **On-demand:** Ideale per carichi imprevedibili, scala rapidamente e si paga per singola richiesta
- **Provisioned:** L'utente prealloca un numero preciso di unità di lettura (RCU) e scrittura (WCU) È più economico per carichi di lavoro costanti e prevedibili